

[Dashboard](#) / [My courses](#) / [ITB IF1210 2 2526](#) / [Praktikum 9](#) / [ADT Binary Search Tree - Praktikum \(STI\)](#).

Started on	Friday, 15 May 2026, 3:35 PM
State	Finished
Completed on	Friday, 15 May 2026, 4:37 PM
Time taken	1 hour 2 mins
Grade	300.00 out of 300.00 (100%)

Question **1**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: bst_tambahan.c

Di Mystery Shack, Dipper ingin menambahkan beberapa fitur analisis pada pohon pencarian biner (**Binary Search Tree**) yang sudah ia miliki. Fitur tersebut harus bisa mengecek keberadaan sebuah nilai pada BST, menghitung jumlah node daun, dan menjumlahkan nilai key pada semua daun.

Tugas Anda adalah mengimplementasikan fungsi tambahan BST pada [bst_tambahan.h](#) agar dapat:

- mengecek apakah sebuah nilai sudah ada pada BST,
- menghitung banyaknya node daun,
- menjumlahkan nilai key pada semua node daun.

Anda hanya diminta mengumpulkan **bst_tambahan.c**. Disarankan memanfaatkan header [bst.h](#) dan [boolean.h](#) bila diperlukan.

Untuk pengujian, Anda boleh membuat driver sendiri. Anda juga dapat memakai program uji ([main.c](#)) yang disediakan sebagai referensi.

Keluaran yang diharapkan dari program uji ([main.c](#)) yang disediakan:

IsInTree 6: YES

IsInTree 5: NO

Leaf count: 4

Leaf sum: 25

C

 [bst_tambahan.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.00 sec, 1.71 MB
2	20	Accepted	0.00 sec, 1.71 MB
3	20	Accepted	0.00 sec, 1.54 MB
4	20	Accepted	0.00 sec, 1.66 MB
5	20	Accepted	0.00 sec, 1.66 MB

Question **2**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: inverseBST.c

Di Nangor Shack, Dapper menemukan bahwa sistem penyimpanan benda misterius milik Gord tersusun dalam bentuk **Binary Search Tree (BST)**. Awalnya, benda-benda tersebut tersusun dari nilai energi terkecil ke terbesar jika ditelusuri secara inorder.

Namun, Mebel tidak sengaja menekan tombol aneh bertuliskan "Reverse Reality" pada mesin Mystery Sorter 5000. aKIBATNYA, URUTAN PENYIMPANAN PERLU DIBALIK AGAR PENELUSURAN INDORDER MENGHASILKAN NILAI DARI YANG TERBESAR KE YANG TERKECIL.

Untuk memperbaiki sistem tersebut, Dapper meminta bantuan kalian untuk mengimplementasikan fungsi `inverseTree()`. Fungsi ini perlu mengubah susunan Mystery Tree agar hasil penelusuran yang sebelumnya bergerak dari energi terkecil ke terbesar menjadi dari energi terbesar ke terkecil.

Implementasikanlah fungsi `inverseTree()` pada [inverseBST.h](#). Setelah fungsi dijalankan, pohon yang awalnya menghasilkan urutan kecil ke besar saat traversal inorder akan berubah menjadi besar ke kecil.

Ketentuan:

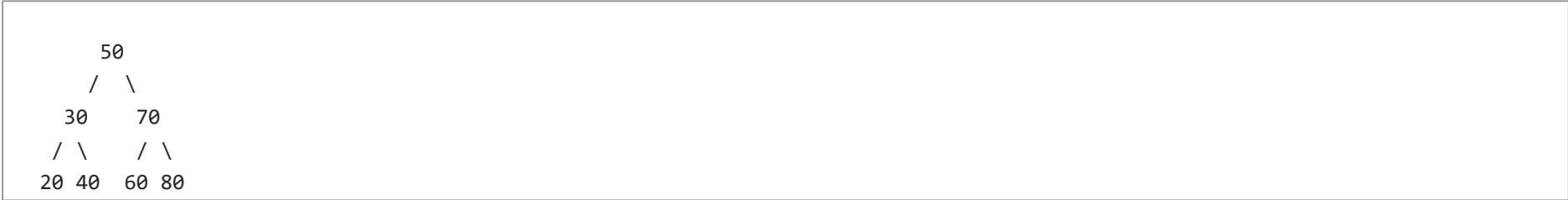
- Jika pohon kosong, fungsi tidak melakukan apa pun.

Fungsi yang harus diimplementasikan:

```
void inverseTree(BinTree *p);
```

Contoh:

Sebelum `inverseTree()`:



Setelah `inverseTree()`:



Anda hanya diminta mengumpulkan `inverseBST.c`. Disarankan memanfaatkan header [bst.h](#), [bintree_traversal.h](#), dan [boolean.h](#) bila diperlukan.

C

[inverseBST.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.00 sec, 1.71 MB
2	20	Accepted	0.00 sec, 1.71 MB

No	Score	Verdict	Description
3	20	Accepted	0.00 sec, 1.50 MB
4	20	Accepted	0.00 sec, 1.59 MB
5	20	Accepted	0.00 sec, 1.63 MB

Question **3**

Correct

Mark 100.00 out of 100.00

Time limit	1 s
Memory limit	64 MB

Nama File: rangeSumBST.c

Setelah Mystery Sorter 5000 berhasil diperbaiki, Gord menemukan fitur lama yang belum sempat ia selesaikan, yaitu **Range Energy Scanner**. Fitur ini digunakan untuk menghitung total energi dari semua benda misterius yang memiliki nilai dalam rentang tertentu.

Dapper ingin mengetahui jumlah seluruh nilai energi benda yang berada di antara dua batas, yaitu **L** dan **R**. Karena data benda sudah disimpan dalam bentuk **Binary Search Tree (BST)**, proses pencarian dapat dilakukan lebih efisien dengan memanfaatkan properti BST.

Dapper ingin mengetahui jumlah seluruh nilai energi benda yang berada di antara dua batas, yaitu **L** dan **R**. Karena data benda sudah disimpan dalam bentuk **Binary Search Tree (BST)**, struktur pohon dapat dimanfaatkan agar proses pencarian dilakukan dengan lebih efisien.

Tantangan kalian adalah mengimplementasikan fungsi untuk mencari jumlah semua elemen dengan nilai key berada dalam range **[L, R]**.

Ketentuan:

- Range bersifat inklusif, sehingga nilai **L** dan **R** ikut dihitung jika ditemukan.
- Jika pohon kosong, hasilnya adalah **0**.
- Jika sebuah node memiliki **count** lebih dari 1, maka kontribusi nilainya adalah **key * count**.

Implementasikanlah fungsi **rangeSumBST()** pada [rangeSumBST.h](#). Setelah fungsi dijalankan, pohon yang awalnya menghasilkan urutan kecil ke besar saat traversal inorder akan berubah menjadi besar ke kecil.

int rangeSumBST(BinTree p, int L, int R);

Contoh:

Isi BST:

20 30 40 50 60 70 80

Jika:

L = 30
R = 70

Maka node yang dihitung adalah:

30 + 40 + 50 + 60 + 70

Sehingga outputnya:

250

Anda hanya diminta mengumpulkan **rangeSumBST.c**. Disarankan memanfaatkan header [bst.h](#), [bintree traversal.h](#), dan [boolean.h](#) bila diperlukan.

C

 [rangeSumBST.c](#)

Score: 100

Blackbox

Score: 100

Verdict: Accepted

Evaluator: Exact

No	Score	Verdict	Description
1	20	Accepted	0.00 sec, 1.50 MB

No	Score	Verdict	Description
2	20	Accepted	0.00 sec, 1.62 MB
3	20	Accepted	0.00 sec, 1.65 MB
4	20	Accepted	0.00 sec, 1.66 MB
5	20	Accepted	0.00 sec, 1.64 MB

[◀ ADT Binary Search Tree - Pra Praktikum \(STI\)](#)

Jump to...

[ADT Binary Search Tree - Latihan Praktikum \(STI\) ▶](#)